Sammy Al Hashemi

sammy.alhashemi@mail.utoronto.ca

(647)-463-2638

github.com/Sammyalhashe

Education

- University of Toronto
  - Bachelor of Applied Science and Engineering
- Department of Engineering Science
- Major in physics

cGPA: 3.56

Software Skills

- Python
- Java
- Angular
- Tensorflow/Keras
- openCV

Hardware Skills

- Verilog

Courses Taken

- Algorithms/Foundations of Computing
- Data Structures & Algorithms
- Machine Learning and Data Mining
- Electronic Circuits/Digital Electronics

Website

https://sammyalhashemi.com

Work Experience

Device Characterization and Product Engineering Intern

Microsemi Corp.

July 2017 - August 2018

- Characterized the Mathblock component of Microsemi's G5 FPGA product as well as performance-related AC tests on other combinatorial and sequential elements found in the FPGAs that went to market.
  - Example components include look-up tables, carry chains, and digital I/O's, characterized mainly for max frequency on different corners of the FPGA fabric.
- Inside Libero, I created Verilog test structures, making use of JTAG to read input from test programs created using a company code.
- Wrote sections of the characterization report found in the official product manual used to facilitate customer knowledge in our device.

Projects

Thesis

- Using IBM Q's python SDK, `qiskit`, I investigated the scalability and applicability of Grover's algorithm for my thesis under Proffessor Glen Gulak.
- My methodology included programming theoretical scalable applications of Grover's Algorithm to real-world scenarios.
- **Report**: https://docdro.id/pd5FKYN

Data Charter

- Built a charting application that reads data that is interfaced from a *ni daqmx* board.
- The actual chart was built using python, `RxPY` and `pyqt5` and is able to display any number of channels vs. time/other channel.
- In production at UofT's Advanced Physics Lab.

Speed Challenge

- Given a single dashcam video as training data, the task was to build a model that could determine the speed of a car.
- Using openCV, I split the data into frames and performed optical flow in a pre-processing step.
- Built a convolutional network based on an NVIDIA paper for their self driving cars.
- Project is still in progress, next steps include splitting the data into testing and validation sets and tune any extra parameters.

DashCam Speed Visualizer

- Using 527 trips of Comma.ai test data, stored as `JSON`, I created a performant webapp that simultaneously displayed all 527 and their speed distribution on an interactive map. See https://dashcamroutes.herokuapp.com/
- I stored the trips in a Mongo Cloud Atlas database in three collections, all dependent on the zoom of the map.
  - Python was used to seperate these trips and uploaded them to the Cloud.
  - Client side caching was implemented so that the data did not need to be downloaded multiple times.
- An `Angular2+` frontend was implemented with an `ExpressJS` backend with `Mongoose` acting as the interface between Mongo and the app.